CS 221: Artificial Intelligence: Principles and Techniques

Assignment 4: Controlling MountainCar

March 6, 2026

SUNet ID:	[your SUNet ID]
Name:	[your first and last name]
Collaborators:	[list all the people you worked with]

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.

Before you get started, please read the Assignments section on the course website thoroughly.

Markov decision processes (MDPs) can be used to model situations with uncertainty (which is most of the time in the real world). In this assignment, you will implement algorithms to find the optimal policy, both when you know the transitions and rewards (value iteration) and when you don't (reinforcement learning). You will use these algorithms on Mountain Car, a classic control environment where the goal is to control a car to go up a mountain.

Problem 1: Value Iteration

In this problem, you will perform the value iteration updates manually on a basic game to build your intuitions about solving MDPs. The set of possible states in this game is States $= \{-2, -1, 0, +1, +2\}$ and the set of possible actions is Actions$(s) = \{a_1, a_2\}$. The starting state is 0 and there are two end states, -2 and $+2$. Recall that the transition function $T : \text{States} \times \text{Actions} \to \Delta(\text{States})$ encodes the probability of transitioning to a next state s' after being in state s and taking action a as $T(s'|s, a)$. In this MDP, the transition dynamics are given as follows:

$\forall i \in \{-1, 0, 1\} \subseteq \text{States}$,

- $T(i-1|i, a_1) = 0.8$ and $T(i+1|i, a_1) = 0.2$
- $T(i-1|i, a_2) = 0.7$ and $T(i+1|i, a_2) = 0.3$

Think of this MDP as a chain formed by states $\{-2, -1, 0, +1, +2\}$. In words, action a_1 has an 80% chance of moving the agent backwards in the chain and a 20% chance of moving the agent forward. Similarly, action a_2 has a 70% chance of sending the agent backwards and a 30% chance of moving the agent forward. We will use a discount factor $\gamma = 1$.

The reward function for this MDP is $\text{Reward}(s, a, s') = \begin{cases} 10 & s' = -2 \\ 50 & s' = +2 \\ -5 & \text{otherwise} \end{cases}$.

a. [3 points] What is the value of $V_i^\star(s)$ for each $s \in \text{States}$ after each iteration $i = \{1, 2\}$ of value iteration? Please write down the values from all iterations. Recall that $\forall s \in \text{States}$, $V_0^\star(s) = 0$ and, for any end state s_{end}, $V_i^\star(s_{\text{end}}) = 0$.

What we expect: The $V_i^\star(s)$ of all 5 states after each iteration. In total, 10 values should be reported. Show your work briefly.

Your Solution:
Initially $V_0(s) = 0$ for all states and the terminal states -2 and $+2$ always have value 0.
Iteration 1
For $s = -1$
$a_1 : 0.8(10) + 0.2(-5) = 7$
$a_2 : 0.7(10) + 0.3(-5) = 5.5$
So $V_1(-1) = 7$.
For $s = 0$
Both actions give reward -5, so $V_1(0) = -5$.
For $s = 1$
$a_1 : 0.8(-5) + 0.2(50) = 6$
$a_2 : 0.7(-5) + 0.3(50) = 11.5$
So $V_1(1) = 11.5$.
Thus

$$V_1(-2) = 0,\ V_1(-1) = 7,\ V_1(0) = -5,\ V_1(1) = 11.5,\ V_1(2) = 0$$

Iteration 2
For $s = -1$
$a_1 : 0.8(10 + 0) + 0.2(-5 - 5) = 6$
$a_2 : 0.7(10 + 0) + 0.3(-5 - 5) = 4$
So $V_2(-1) = 6$.
For $s = 0$
$a_1 : 0.8(-5 + 7) + 0.2(-5 + 11.5) = 2.9$
$a_2 : 0.7(-5 + 7) + 0.3(-5 + 11.5) = 3.35$
So $V_2(0) = 3.35$.
For $s = 1$
$a_1 : 0.8(-5 - 5) + 0.2(50) = 2$
$a_2 : 0.7(-5 - 5) + 0.3(50) = 8$
So $V_2(1) = 8$.
Thus

$$V_2(-2) = 0,\ V_2(-1) = 6,\ V_2(0) = 3.35,\ V_2(1) = 8,\ V_2(2) = 0$$

b. [1 point] Using $V_2^\star(\cdot)$, what is the corresponding optimal policy $\pi^\star$ for all non-end states?

What we expect: Optimal policy $\pi^\star(s)$ for each non-end state. Show your work briefly.

Your Solution: From V_2 we pick the action with the higher value.

For $s = -1$: $a_1(6) > a_2(4)$ so $\pi^*(-1) = a_1$.

For $s = 0$: $a_2(3.35) > a_1(2.9)$ so $\pi^*(0) = a_2$.

For $s = 1$: $a_2(8) > a_1(2)$ so $\pi^*(1) = a_2$.

Problem 2: Transforming MDPs

In computer science, the idea of a reduction is very powerful: say you can solve problems of type A, and you want to solve problems of type B. If you can convert (reduce) any problem of type B to type A, then you automatically have a way of solving problems of type B potentially without doing much work! We saw an example of this for search problems when we reduce A* to UCS. Now let's do it for solving MDPs.

a. [4 points] Suppose we have an MDP with states States and a discount factor $\gamma < 1$, but we have an MDP solver that can only solve MDPs with discount factor of 1. How can we leverage this restricted MDP solver to solve the original MDP?

Let us define a new MDP with states $\text{States}' = \text{States} \cup \{o\}$, where o is a new state. Let's use the same actions $\text{Actions}'(s) = \text{Actions}(s)$ but keep the discount $\gamma' = 1$. Your job is to define new transition probabilities $T'(s'|s,a)$ and rewards $\text{Reward}'(s,a,s')$ in terms of the original MDP such that the optimal values $V_{\text{opt}}(s)$ for all $s \in \text{States}$ are equal under the original MDP and the new MDP.

HINT: What recurrence must the optimal values for each MDP satisfy? You can show that the optimal values for each MDP are the same if these recurrences are equivalent. If you're not sure how to approach this problem, see the MDP lecture slides on convergence.

What we expect: Transition probabilities (T') and reward function (Reward') written in mathematical expressions, followed by a short verification to show that the two optimal values are equal. Use consistent notation from the question. Show your work briefly.

Your Solution:

We want to convert the original MDP with discount factor $\gamma < 1$ into a new MDP with $\gamma' = 1$.

We add a new state o and define the new states as

$$\text{States}' = \text{States} \cup \{o\}$$

For transitions, we modify the probabilities as follows:

$$T'(s'|s,a) = \gamma\, T(s'|s,a) \quad \text{for } s' \in \text{States}$$

$$T'(o|s,a) = 1 - \gamma$$

State o is an absorbing state:

$$T'(o|o,a) = 1$$

The reward function stays the same for the original transitions:

$$Reward'(s, a, s') = Reward(s, a, s')$$

and

$$Reward'(s, a, o) = 0$$

Verification

Original MDP:

$$V(s) = \max_a \sum_{s'} T(s'|s, a) \left[Reward(s, a, s') + \gamma V(s')\right]$$

New MDP:

$$V'(s) = \max_a \sum_{s'} T'(s'|s, a) \left[Reward'(s, a, s') + V'(s')\right]$$

Substituting T':

$$V'(s) = \max_a \sum_{s'} \gamma T(s'|s, a) \left[Reward(s, a, s') + V'(s')\right]$$

which is equivalent to the original Bellman equation, so

$$V'(s) = V(s)$$

for all $s \in$ States.

Problem 3: Value Iteration on Mountain Car

Now that we have gotten a bit of practice with general-purpose MDP algorithms, let's use them for some control problems. Mountain Car is a classic example in robot control where you try to get a car to the goal located on the top of a steep hill by accelerating left or right. We will use the implementation provided by The Farama Foundation's Gymnasium, formerly OpenAI Gym.

The state of the environment is provided as a pair (position, velocity). The starting position is randomized within a small range at the bottom of the hill. At each step, the actions are either to accelerate to the left, to the right, or do nothing, and the transitions are determined directly from the physics of a frictionless car on a hill. Every step produces a reward based on the car's distance from the goal and velocity.

Note: We are using Python 3.12 for this assignment. To get the feel for the environment, test with an untrained agent which takes a random action at each step:

```
uv run mountaincar.py --agent naive
```

You will see the agent struggling, not able to complete the task within the time limit. In this assignment, you will train this agent with different reinforcement learning algorithms so that it can learn to climb the hill. As the first step, we have designed two MDPs for this task. The first uses the car's continuous (position, velocity) state as is, and the second discretizes the position and velocity into bins and uses indicator vectors.

Carefully examine `ContinuousGymMDP` and `DiscreteGymMDP` classes in `util.py` and make sure you understand.

If we want to apply value iteration to the `DiscreteGymMDP` (think about why we can't apply it to `ContinuousGymMDP`), we require the transition probabilities $T(s, a, s')$ and rewards $R(s, a, s')$ to be known. But oftentimes in the real world, T and R are unknown, and the gym environments are set up in this way as well, only interfacing through the `.step()` function. One method to still determine the optimal policy is model-based value iteration, which runs Monte Carlo simulations to estimate $\hat{T}$ and $\hat{R}$, and then runs value iteration. This is an example of model-based RL. Examine `RLAlgorithm` in `util.py` to understand the `get_action` and `incorporate_feedback` interface and peek into the `simulate` function to see how they are called repeatedly when training over episodes.

a. [5 points] As a warm up, implement `value_iteration` as you learned in lectures, and run it on the number line MDP from Problem 1. The inputs are dense NumPy tensors for transition probabilities and rewards of shape `(num_states, num_actions, num_states)` with an optional mask of valid actions. Return a NumPy array of optimal action identifiers per state (use `None` when no action is available).

   What we expect: An implementation of `value_iteration` in `submission.py`.

b. [10 points] In `submission.py`, implement `ModelBasedMonteCarlo`, which runs `value_iteration`

every `calc_val_iter_every` steps. Maintain NumPy arrays for transition counts, accumulated rewards, and a boolean mask of visited actions. Use MDP helpers (e.g., `state_to_index`) to map between raw states and indices. Implement `get_action` and `incorporate_feedback`.

What we expect: An implementation of `ModelBasedMonteCarlo` in `submission.py`.

c. [2 points] Run `uv run train.py --agent value-iteration` to train the agent and view reward-per-episode plots (three trials). Comment on the plots and discuss when model-based value iteration could perform poorly. You can also run `uv run mountaincar.py --agent value-iteration` to visualize performance.

What we expect: Plots of rewards and 2–3 sentences describing the plots and when model-based value iteration may fail.

Your Solution: From the plots, the reward generally increases over episodes, which shows that the agent is learning a better policy over time. At the beginning the rewards are very low because the agent does not know how to reach the goal, but after more interactions the performance improves as the transition and reward estimates become more accurate. Model-based value iteration may perform poorly when the environment is very large or complex because estimating the transition probabilities accurately can require many samples.

Problem 4: Q-Learning Mountain Car

In the previous question, we've seen how value iteration can take an MDP which describes the full dynamics of the game and return an optimal policy, and we've also seen how model-based value iteration with Monte Carlo simulation can estimate MDP dynamics if unknown at first and then learn the respective optimal policy. But suppose you are trying to control a complex system in the real world where trying to explicitly model all possible transitions and rewards is intractable. We will see how model-free reinforcement learning can nevertheless find the optimal policy.

a. [10 points] For a discretized MDP, we have a finite set of `(state, action)` pairs. We learn the Q-value for each pair using the Q-learning update. In `TabularQLearning`, the Q-table is a NumPy array of shape `(num_states, num_actions)`. Implement `get_action` (epsilon-greedy using `exploration_prob`) and `incorporate_feedback` (update the appropriate entry using the provided indices; no Python dictionaries).

What we expect: Implementations of `get_action` and `incorporate_feedback` in `TabularQLearning`.

b. [5 points] For Q-learning in continuous states, we use function approximation. We will use a Fourier feature extractor. For state $s = (s_1, \ldots, s_k)$, maximum coefficient m, and optional per-dimension `scale`, the feature extractor returns all terms of the form $\cos\big(\pi \sum_{i=1}^{k} c_i\, d_i s_i\big)$ for $c_i \in \{0, \ldots, m\}$ and scaling d_i. Concretely for $k = 2$ and $m = 2$, this corresponds to the "outer-sum" grid of combinations before applying $\cos(\pi\cdot)$.

Implement `fourier_feature_extractor` in `submission.py`. (Looking at `util.polynomial_feature_` may help with NumPy broadcasting.)

What we expect: An implementation of `fourier_feature_extractor` in `submission.py`.

c. [10 points] With function approximation, Q-values are computed as a dot product of features and learned weights. Using `fourier_feature_extractor` from (b), complete `FunctionApproxQLearning`.

What we expect: An implementation of `FunctionApproxQLearning` using `fourier_feature_extractor`.

d. [2 points] Run `uv run train.py --agent tabular` and `uv run train.py --agent function-approximation` to see reward plots (three trials each) and comment on the results. If none of your plots are improving, double-check `incorporate_feedback`. You can also run `uv run mountaincar.py --agent tabular` and `uv run mountaincar.py --agent function-approximation` to visualize.

What we expect: Plots and 2–3 sentences comparing `TabularQLearning` and `FunctionApproxQLearning`; discuss possible reasons.

Your Solution:

From the plots, tabular Q-learning generally reaches better rewards compared to function approximation. The tabular method stores an exact Q-value for each state-action pair in the discretized state space, which makes learning more stable for this environment. Function approximation performs slightly worse because the Fourier features only approximate the Q-function, so the learned values may not perfectly represent the optimal policy.

e. [2 points] Despite sometimes worse performance here, why can function-approximation Q-learning outperform tabular Q-learning in other environments? Consider limited exploration, very high-dimensional state spaces, and/or space constraints.

What we expect: 2–3 sentences discussing why `FunctionApproxQLearning` can be better in various scenarios.

Your Solution:

Function approximation can perform better in environments with very large or continuous state spaces where storing a full Q-table is not practical. It can generalize across similar states, which allows the agent to learn useful behaviors even with limited exploration. It is also more memory efficient because it stores only a set of weights instead of values for every state-action pair.

Problem 5: Safe Exploration

We learned about different state exploration policies for RL in order to get information about `(s,a)`. The method implemented in our MDP code is epsilon-greedy exploration, which balances both exploitation (choosing the action a that maximizes $\hat{Q}_{\text{opt}}(s,a)$) and exploration (choosing the action a randomly):

$$\pi_{\text{act}}(s) = \begin{cases} \arg\max_{a \in \text{Actions}} \hat{Q}_{\text{opt}}(s,a) & \text{probability } 1-\epsilon \\ \text{random from Actions}(s) & \text{probability } \epsilon \end{cases}$$

In real-life scenarios when safety is a concern, there might be constraints set during state exploration. Safe exploration in RL is a critical research question in AI safety and human–AI interaction.

Assume there are harmful consequences for the driver of the Mountain Car if the car exceeds a certain velocity. One simple approach is to restrict the set of potential actions at each step to avoid unsafe speeds.

a. [2 points] The Mountain Car MDP includes velocity constraints via a `self.max_speed` parameter. Read OpenAI Gym's Mountain Car implementation and explain how `self.max_speed` is used.

What we expect: One sentence on how `self.max_speed` is used in `mountain_car.py`.

Your Solution:

The parameter `self.max_speed` limits the velocity of the car by clipping the velocity value after each update so that it stays within the allowed range $[-\text{max_speed}, \text{max_speed}]$.

b. [1 point] Run Function Approximation Q-Learning without the `max_speed` constraint: `uv run train.py --agent function-approximation --max_speed=100000`. We approximate removing the constraint by setting a very large value. Ignoring reward scale differences (rewards scale with `max_speed`), notice the learned behavior may not change much. Explain in 1–2 sentences why.

What we expect: 1–2 sentences explaining why the Q-Learning result doesn't necessarily change.

Your Solution:

Even without the max speed constraint, the learned policy stays mostly the same because the agent rarely reaches very high velocities during normal training. The optimal strategy for solving the Mountain Car task does not require extremely large speeds, so removing the limit does not significantly change the behavior.

c. [2 points] Constrain the action set directly. In `ConstrainedQLearning`, implement action constraints so that `velocity_(t+1) < velocity_threshold`. If no actions are valid, return `None`. Mountain Car updates velocity as:

$$\text{velocity}_{t+1} = \text{velocity}_t + (\text{action} - 1) * \text{force} - \cos(3 * \text{position}_t) * \text{gravity}.$$

Treat 0.065 as unsafe. After implementing the constraint, run: `uv run grader.py 5c-helper` to compare policies for two continuous MDPs (one with `max_speed` = 0.065 and one very large). Explain in 1–2 sentences why the output policies differ now.

What we expect: Complete `ConstrainedQLearning` in `submission.py`, then run `uv run grader.py 5c-helper`. Include 1–2 sentences explaining the difference.

Your Solution:

When the velocity constraint is applied through the action space, some actions are no longer allowed if they would lead to unsafe speeds. Because of this restriction, the agent must choose different actions to stay within the allowed velocity range, which leads to a different learned policy compared to the unconstrained case.

d. [2 points] **Designing safe exploration in the real world.** Consider autonomous vehicles trained with RL on public roads in residential neighborhoods. Design two MDPs and exploration strategies:

- **MDP A (unsafe):** Define States, Actions(s), and Reward(s, a, s') and an exploration policy where the vehicle could cause harm (e.g., endangering pedestrians, breaking traffic laws).
- **MDP B (safer):** Modify MDP A to reduce or avoid the identified harms (e.g., constraints, penalties, restricted exploration).

For each MDP, include States, Actions(s), Reward(s, a, s'), the exploration policy, and a brief (1–2 sentence) ethical explanation of how the design could cause or mitigate harm.

What we expect: Definitions of two MDPs (states, actions, rewards), their exploration policies, and 1–2 sentence explanations of how their design could cause or mitigate harm.

Your Solution:

MDP A

States: The state includes the car's position, speed, distance to nearby pedestrians, and traffic signal status.

Actions: accelerate, brake, turn left, turn right, or maintain speed.

Reward: positive reward for reaching the destination quickly, small penalty for slowing down, and negative reward for collisions or traffic violations.

Exploration policy: epsilon-greedy exploration.

Ethical concern: Random exploration may cause unsafe actions such as driving too fast near pedestrians or ignoring traffic signals.

MDP B

States: Same as MDP A.

Actions: Only actions that keep the car within safe speed limits and safe distance from pedestrians are allowed.

Reward: positive reward for safe driving and reaching the destination, with large penalties for unsafe behavior.

Exploration policy: constrained exploration where unsafe actions are removed from the action set.

Ethical improvement: By restricting unsafe actions, the agent avoids harmful behaviors during training and reduces the risk to pedestrians and other drivers.